Note de la séance de remédiation SQL

Fenêtrage, parts, comptages avec LEFT JOIN, jointures, consignes | 2026-10-21

1. Les cinq règles de la séance

Notion	Règle à retenir
Classement (fenêtrage)	Dans OVER, PARTITION BY définit les groupes, ORDER BY le critère de classement. Sans DESC, le ran
Part dans un groupe	SUM(x) OVER (PARTITION BY groupe) met le total du groupe sur chaque ligne. OVER () ou une sous-re
Comptage avec LEFT JOIN	COUNT(o.id) et non COUNT(*). Ne jamais regrouper sur un montant. Pour filtrer sans perdre de lignes, la
Jointures et cohérence	Chaque ON relie la table ajoutée à une table déjà présente. Après chaque jointure, compter les lignes ; la
Consignes	Lister ce que l'énoncé demande en sortie (colonnes, valeurs 0, tri, format, nombre de CTE) et vérifier cha

2. Exercices et corrections

A1 : Pour chaque ville, le client le plus jeune

```
WITH r AS (
    SELECT *, RANK() OVER (PARTITION BY ville ORDER BY age) AS rang FROM clients
)
SELECT prenom, ville, age FROM r WHERE rang = 1;
```

Résultat : Marie (Kinshasa 22), Jean (Lubumbashi 35), David (Goma 26)

A2 : Pour chaque ville, le client le plus âgé

```
WITH r AS (
    SELECT *, RANK() OVER (PARTITION BY ville ORDER BY age DESC) AS rang FROM clients
)
SELECT prenom, ville, age FROM r WHERE rang = 1;
```

Résultat : Eric (45), Sarah (38), Patrick (41)

A3 : Commandes classées par montant décroissant dans chaque catégorie

```
SELECT p.nom, p.categorie, o.montant,
       RANK() OVER (PARTITION BY p.categorie ORDER BY o.montant DESC) AS rang
FROM commandes o
INNER JOIN produits p ON o.produit_id = p.id;
```

Résultat : Accessoires : Montre 1, Casque 2, Clavier 3, Souris 4 ; Electronique : Ordinateur 1, Tablette 2, Telephone 3

A4 : Commande la plus récente de chaque client

```
WITH r AS (
    SELECT c.id, c.prenom, p.nom, o.date_commande,
           RANK() OVER (PARTITION BY c.id ORDER BY o.date_commande DESC) AS rang
    FROM clients c
    INNER JOIN commandes o ON c.id = o.client_id
    INNER JOIN produits p ON p.id = o.produit_id
)
SELECT prenom, nom, date_commande FROM r WHERE rang = 1;
```

Résultat : Awa Casque, Jean Ordinateur, Marie Clavier, Grace Tablette

B1 : Part de chaque commande dans la dépense du client

```
SELECT c.prenom, o.id AS id_commande, o.produit, o.montant,
       ROUND(o.montant * 100.0 / SUM(o.montant) OVER (PARTITION BY c.id), 2) AS pct_client
FROM commandes o
INNER JOIN clients c ON o.client_id = c.id;
```

Résultat : Awa 86,21 et 13,79 ; Marie 33,33 et 66,67 ; Jean 100 ; Grace 100

B2 : Part de chaque commande dans le total de la province

```
SELECT c.prenom, o.produit, v.province, o.montant,
       ROUND(o.montant * 100.0 / SUM(o.montant) OVER (PARTITION BY v.province), 2) AS pct_province
FROM commandes o
INNER JOIN clients c ON o.client_id = c.id
INNER JOIN villes v ON c.ville = v.ville;
```

Résultat : Kinshasa : 39,37 / 6,30 / 2,36 / 4,72 / 47,24 ; Haut-Katanga : 100

C1 : Nombre de commandes et plus grosse commande par client (0 si aucune)

```
SELECT c.id, c.prenom, COUNT(o.id) AS nb_commandes,
       COALESCE(MAX(o.montant), 0) AS plus_grosse
FROM clients c
LEFT JOIN commandes o ON o.client_id = c.id
GROUP BY c.id, c.prenom;
```

Résultat : Awa 2 et 250, Jean 1 et 800, Marie 2 et 30, Grace 1 et 300, les 4 autres 0 et 0

C2 : Par ville de la table villes : commandes de plus de 100 (la condition va dans le ON)

```
SELECT v.ville,
       COUNT(o.id) AS nb_commandes,
       COALESCE(SUM(o.montant), 0) AS montant_total
FROM villes v
LEFT JOIN clients c ON c.ville = v.ville
LEFT JOIN commandes o ON o.client_id = c.id AND o.montant > 100
GROUP BY v.ville;
```

Résultat : Goma 0 et 0, Kinshasa 2 et 550, Lubumbashi 1 et 800

D1 : Commandes par province et catégorie, avec vérification de la somme

```
SELECT v.province, p.categorie, COUNT(o.id) AS nb_commandes
FROM commandes o
INNER JOIN clients c ON o.client_id = c.id
INNER JOIN villes v ON c.ville = v.ville
INNER JOIN produits p ON o.produit_id = p.id
GROUP BY v.province, p.categorie;

-- Contrôle : la somme des comptes doit égaler ce nombre
SELECT COUNT(*) FROM commandes o INNER JOIN clients c ON o.client_id = c.id;
```

Résultat : Kinshasa Accessoires 3, Kinshasa Electronique 2, Haut-Katanga Electronique 1 ; contrôle : 6

D2 : Total des commandes de clients connus et somme des totaux par province

```
WITH totaux_province AS (
    SELECT v.province, SUM(o.montant) AS total
    FROM clients c
    INNER JOIN commandes o ON c.id = o.client_id
    INNER JOIN villes v ON c.ville = v.ville
    GROUP BY v.province
)
SELECT (SELECT SUM(o.montant) FROM commandes o
        INNER JOIN clients c ON o.client_id = c.id) AS total_clients_connus,
       (SELECT SUM(total) FROM totaux_province) AS somme_provinces;
```

Résultat : 1435 et 1435 (la commande orpheline de 120 explique 1555 au total)

D3 : Jointure fausse (c.ville = c.ville), corrigée

```
SELECT v.province, SUM(o.montant) AS total
FROM clients c
INNER JOIN commandes o ON c.id = o.client_id
INNER JOIN villes v ON c.ville = v.ville
GROUP BY v.province;
```

Résultat : Kinshasa 635, Haut-Katanga 800 (la version fausse donnait 1435 pour chaque province)

E1 : Deux CTE enchaînés : provinces au-dessus de la moyenne

```
WITH prov_tot AS (
    SELECT v.province, SUM(o.montant) AS total
    FROM clients c
    INNER JOIN commandes o ON c.id = o.client_id
    INNER JOIN villes v ON c.ville = v.ville
    GROUP BY v.province
),
moyenne_prov AS (
    SELECT AVG(total) AS moyenne FROM prov_tot
)
SELECT * FROM prov_tot WHERE total > (SELECT moyenne FROM moyenne_prov);
```

Résultat : Haut-Katanga 800

E2 : Commandes de plus de 30, date au format JJ/MM/AAAA, de la plus récente à la plus ancienne

```
SELECT id, produit, montant, DATE_FORMAT(date_commande, '%d/%m/%Y') AS date_fr
FROM commandes
WHERE montant > 30
ORDER BY date_commande DESC;
```

Résultat : Montre 02/04, Tablette 18/03, Ordinateur 10/02, Casque 28/01, Telephone 15/01

F1 : Par province : nombre de commandes et montant total (0 si aucune)

```
SELECT v.province,
       COUNT(o.id) AS nb_commandes,
       COALESCE(SUM(o.montant), 0) AS montant_total
FROM villes v
LEFT JOIN clients c ON c.ville = v.ville
LEFT JOIN commandes o ON c.id = o.client_id
GROUP BY v.province;
```

Résultat : Haut-Katanga 1 et 800, Kinshasa 5 et 635, Nord-Kivu 0 et 0

F2 : Par ville : le client qui a le moins dépensé

```
WITH commande_range AS (
    SELECT c.id, c.prenom, c.ville, SUM(o.montant) AS total,
           RANK() OVER (PARTITION BY c.ville ORDER BY SUM(o.montant)) AS rang
    FROM clients c
    INNER JOIN commandes o ON c.id = o.client_id
    GROUP BY c.id, c.prenom, c.ville
)
SELECT ville, prenom, total FROM commande_range WHERE rang = 1;
```

Résultat : Kinshasa Marie 45, Lubumbashi Jean 800

F3 : Par catégorie : part de chaque province

```
WITH prov_cat AS (
    SELECT v.province, p.categorie, SUM(o.montant) AS total
    FROM commandes o
    INNER JOIN clients c ON o.client_id = c.id
    INNER JOIN villes v ON c.ville = v.ville
    INNER JOIN produits p ON p.id = o.produit_id
    GROUP BY v.province, p.categorie
)
SELECT province, categorie, total,
       ROUND(total * 100.0 / SUM(total) OVER (PARTITION BY categorie), 2) AS pct_categorie
FROM prov_cat;
```

Résultat : Accessoires Kinshasa 100 ; Electronique Haut-Katanga 59,26, Kinshasa 40,74

3. Pièges rencontrés

• **WHERE après un LEFT JOIN** : `WHERE o.montant > 100` supprime les villes sans commande (Goma). Mettre la condition dans le ON.

• **Mauvaise table de départ** : « chaque ville de la table villes » impose `FROM villes`.

• **Jointure sans lien** : `ON c.ville = c.ville` est toujours vrai, chaque ligne est associée à toutes les villes et chaque province affiche le total général (1435).

• **Alias identique à une colonne** : dans MySQL, ORDER BY préfère l'alias à la colonne. Un alias `date_commande` sur une date formatée trierait sur le texte JJ/MM/AAAA.

• **Contrôle de cohérence** : la commande orpheline (client 9) explique l'écart entre 1555 (toutes les commandes) et 1435 (clients connus).

• **Oublis en sortie** : colonne du client, montant total, valeur 0 : relire la liste des éléments demandés.